BỐI CẢNH, STAKEHOLDER VÀ PHẠM VI HỆ THỐNG

(Context & Scope Specification)

Dự án: Hệ thống Đặt đồ uống trực tuyến BrewLite (Coffee Ordering System)
Học phần: Công nghệ Phần mềm (Software Engineering) – Assignment 2
Công nghệ cốt lõi: Next.js (Mobile-First / PWA), NestJS, PostgreSQL
Phiên bản: 1.0 **Ngày lập:** 20/10/2026

1 Bối cảnh & Vấn đề nghiệp vụ (Business Context & Problem Statement)

1.1 Bối cảnh dự án

Trong mô hình kinh doanh quán cà phê và đồ uống hiện đại, tốc độ phục vụ, tính chính xác của đơn hàng và trải nghiệm cá nhân hóa là yếu tố sống còn để thu hút và giữ chân khách hàng. **BrewLite** là nền tảng quản lý và đặt đồ uống trực tuyến hướng tới trải nghiệm gọn nhẹ, nhanh chóng trên thiết bị di động (*Mobile-First / PWA*), kết nối trực tiếp và đồng bộ giữa Khách hàng, Nhân viên pha chế (*Barista*) và Quản lý cửa hàng.

1.2 Vấn đề nghiệp vụ cần giải quyết (Problem Statement)

Hệ thống giải quyết 5 bài toán thực tế thường gặp tại các cửa hàng cà phê bán lẻ truyền thống:

1. **Quá tải và chờ đợi lâu vào giờ cao điểm:** Khách hàng phải xếp hàng dài để chờ nhân viên ghi nhận món tại quầy, gây gián đoạn và giảm tỷ lệ hài lòng của khách.
2. **Sai sót trong tiếp nhận và pha chế order:** Khách hàng có nhu cầu tùy ý thay đổi yêu cầu (chọn kích cỡ Size S/M/L, chọn nhiều topping kèm theo). Việc truyền đạt trực tiếp hoặc ghi chép giấy dễ dẫn đến nhầm lẫn, làm sai công thức gây lãng phí nguyên liệu.
3. **Thiếu tính minh bạch trong theo dõi tiến độ đơn hàng:** Khách hàng không nắm được đơn của mình đang ở giai đoạn nào (chờ thanh toán, đang pha chế, hay đã xong để lấy), dẫn đến việc tụ tập chờ đợi quanh quầy bar gây lộn xộn.
4. **Quản lý tồn kho và doanh thu phân tán:** Khó kiểm soát tồn kho tức thời (*real-time stock*) dẫn đến tình trạng khách đặt món nhưng thực tế đã hết hàng; thiếu cơ chế đối soát thanh toán và rủi ro đặt trùng đơn do lỗi mạng.
5. **Thiếu cơ chế gắn kết khách hàng (Customer Loyalty):** Cửa hàng chưa có giải pháp tự động tích lũy điểm thưởng thành viên theo giá trị đơn hàng để tái thu hút khách hàng quay lại.

2 Bảng phân tích các bên liên quan (Stakeholder Matrix)

Đề bài Assignment 2 quy định bắt buộc phải phân tích đầy đủ **6 nhóm stakeholder** chính:

Bảng 1: Bảng phân tích 6 nhóm bên liên quan (Stakeholder Matrix)

STT	Stakeholder	Mục tiêu chính (Goals)	Kỳ vọng đối với hệ thống	Mức độ
1	**Khách hàng** *(Customer)*	- Xem menu đồ uống và giá minh bạch. - Tùy chỉnh kích thước và Topping. - Đặt hàng, thanh toán nhanh và nhận thông báo khi đồ uống hoàn thành. - Tích lũy điểm thưởng tự động.	- Giao diện thân thiện, mượt mà trên mobile, tải dưới 2 giây. - Giỏ hàng tính tiền chính xác kèm phí topping. - Nhận diện rõ ràng trạng thái đơn hàng. - Không bị mất phiên đăng nhập đột ngột.	**Rất cao**
2	**Nhân viên quầy / Pha chế** *(Barista)*	- Tiếp nhận danh sách đơn hàng tức thời theo thứ tự thời gian. - Nắm rõ chi tiết từng món (Size, Topping). - Cập nhật nhanh trạng thái pha chế.	- Giao diện trực quan, dễ thao tác trên màn hình quầy. - Phản hồi trạng thái đơn tức thời, tránh trùng hoặc sót đơn. - Giảm thiểu order thủ công tại quầy.	**Cao**
3	**Quản lý cửa hàng** *(Store Manager)*	- Kiểm soát chính xác số lượng tồn kho theo thời gian thực. - Bật/tắt món hoặc topping khi hết nguyên liệu. - Theo dõi dòng tiền và báo cáo lịch sử đơn.	- Hệ thống cảnh báo tự động khi sản phẩm hết hàng hoặc ngừng kinh doanh. - Dữ liệu tài chính nhất quán, bảo mật và lưu vết giao dịch đầy đủ.	**Cao**
4	**Giảng viên / Nghiệm thu** *(Lecturer / Evaluator)*	- Kiểm chứng việc áp dụng đúng quy trình phát triển phần mềm (Agile qua 2 Sprint). - Đánh giá tính hoàn thiện của các tạo phẩm kỹ thuật.	- Tài liệu chuẩn xác, nhất quán với mã nguồn và sơ đồ thiết kế. - Hệ thống có khả năng truy vết hoàn chỉnh (*Traceability Matrix*). - Ứng dụng chạy ổn định trên Production qua HTTPS.	**Rất cao**
5	**Nhóm phát triển** *(Development Team)*	- Xây dựng hệ thống module hóa, dễ mở rộng, mã nguồn sạch sẽ. - Đảm bảo độ tin cậy thông qua kiểm thử tự động. - Tự động hóa đóng gói và triển khai CI/CD.	- Cấu trúc thư mục monorepo/multi-apps rõ ràng. - Cơ chế migration dữ liệu nhất quán với Prisma ORM. - Dễ dàng chạy và tái tạo môi trường local qua Docker Compose.	**Cao**
6	**Đơn vị thanh toán** *(Payment Provider – Mock)*	- Xử lý trừ tiền và xác thực giao dịch an toàn. - Ngăn chặn gian lận hoặc thanh toán trùng lặp (*Idempotency*).	- Phản hồi kết quả thanh toán chính xác (`SUCCESS` hoặc `FAILED`). - Có cơ chế `idempotencyKey` an toàn khi client gửi lại request.	**Trung bình**

3 Phạm vi hệ thống (System Scope)

3.1 Phạm vi trong hệ thống (In-Scope)

3.1.1 A. Phạm vi cốt lõi (MVP – Minimum Viable Product – Sprint 1)

- **Danh mục đồ uống (Menu Browsing):**
  - Khách hàng xem danh sách đồ uống kèm ảnh, giá cơ bản, mô tả và tình trạng còn/hết hàng.
  - API truy vấn danh sách và chi tiết món: `GET /products`, `GET /products/:id`.
  - Giao diện Mobile-first đáp ứng đủ 3 trạng thái bắt buộc: Loading (Skeleton), Empty State và Error Alert.
- **Dữ liệu mẫu & Khởi tạo Schema:** Xây dựng cấu trúc bảng ban đầu và nạp dữ liệu mẫu (*Database Seeding*) cho menu đồ uống trong PostgreSQL.

3.1.2 B. Phạm vi hoàn thiện hệ thống (Sprint 2)

- **Tùy biến món & Giỏ hàng (Khách vãng lai không cần đăng nhập):**
  - Chọn kích cỡ món (Size S cơ sở, Size M +5.000đ, Size L +10.000đ).
  - Chọn thêm Topping (trân châu, thạch, kem cheese) kèm cập nhật giá tức thời.
  - Quản lý giỏ hàng: Thêm món, điều chỉnh số lượng ($1 \leq \textit{quantity} \leq 99$), xóa món và tính tổng tiền tự động.
- **Xác thực & Phân quyền (Authentication & Authorization):**
  - Đăng ký, đăng nhập tài khoản bằng Email và Mật khẩu (mã hóa an toàn bằng `bcrypt`).
  - Cấp phát và xác thực phiên làm việc bằng JSON Web Token (JWT Bearer Token).
  - Phân quyền theo vai trò (*Role-based Access Control*): Khách hàng (`customer`) và Barista (`barista`).
- **Quy trình Đặt hàng & Thanh toán trực tuyến:**
  - Tạo đơn hàng trong một Database Transaction (Atomic) đảm bảo an toàn dữ liệu.
  - Kiểm soát tồn kho đồng thời qua cơ chế Khóa lạc quan (*Optimistic Concurrency Control* bằng cột `version`).
  - Thanh toán trực tuyến (**Bắt buộc đăng nhập**): Hỗ trợ cổng thanh toán mô phỏng Ví điện tử (`WALLET`) và Thẻ (`CARD`).
  - Cơ chế **Idempotency Key** (`idempotencyKey`) chống thanh toán trùng lặp khi người dùng bấm liên tiếp hoặc mạng chập chờn.
- **Theo dõi đơn hàng & Điểm thưởng:**
  - Theo dõi trạng thái đơn hàng thời gian thực theo máy trạng thái: `PENDING` → `PAID` → `PREPARING` → `READY` → `COMPLETED`.
  - Xem danh sách lịch sử các đơn hàng đã đặt và chi tiết từng hóa đơn.
  - Tự động tích lũy điểm thưởng thành viên: 10.000 VNĐ = 1 điểm vào tài khoản.

- **Kiểm thử & Tự động hóa CI/CD:**
  - Unit test cho hàm tính tiền, Integration test cho API tạo đơn/thanh toán, E2E test cho luồng đặt hàng.
  - Pipeline CI trên GitHub Actions: Install → Lint → Unit Test → Build.
  - Đóng gói Docker, file `docker-compose.yml` và triển khai lên Production có HTTPS.

3.2 Phạm vi mở rộng (Future Scope)

- Thông báo đẩy (*Web Push Notifications / WebSocket*) khi đơn hàng chuyển sang trạng thái sẵn sàng nhận (`READY`).
- Hệ thống đánh giá, bình luận và chấm điểm đồ uống sau khi hoàn thành đơn.
- Báo cáo thống kê biểu đồ doanh thu theo ngày/tuần/tháng dành cho quản lý cửa hàng.

3.3 Ngoài phạm vi thực hiện (Out-of-Scope)

Để đảm bảo thời gian và độ sâu kỹ thuật theo yêu cầu đồ án, các nội dung sau **không thực hiện**:

1. **Cổng thanh toán thực tế:** Không liên kết trực tiếp cổng thanh toán Live/Sandbox của MoMo, VNPAY, ZaloPay hay Stripe (sử dụng Mock Payment Service nội bộ).
2. **Giao hàng đường dài:** Không tích hợp định vị GPS hoặc đơn vị vận chuyển thứ ba (GrabExpress, ShopeeFood). Khách nhận đồ uống tại quầy.
3. **Phần cứng quầy POS vật lý:** Không tích hợp máy in hóa đơn nhiệt ESC/POS, máy POS cà thẻ hay két tiền tự động.
4. **Quản lý nhân sự và ca kíp:** Không xây dựng phân hệ chấm công, tính lương nhân viên.

4 Các giả định & Ràng buộc (Assumptions & Constraints)

4.1 Giả định (Assumptions)

1. **Thiết bị người dùng:** Khách hàng sở hữu smartphone hoặc máy tính có trình duyệt web hiện đại hỗ trợ JavaScript/HTML5 và kết nối mạng ổn định.
2. **Quy mô vận hành:** Hệ thống phục vụ cho một cửa hàng cà phê đơn lẻ (*Single Store*) với lưu lượng truy cập dưới 100 giao dịch đồng thời trong giờ cao điểm.
3. **Thanh toán:** Khách hàng luôn có đủ số dư tài khoản khi chọn phương thức ví hợp lệ trong kịch bản kiểm thử luồng chuẩn.
4. **Môi trường đánh giá:** Giảng viên và người chấm bài có thể truy cập được các URL public (frontend, API backend và health check) qua giao thức HTTPS.

4.2 Ràng buộc kỹ thuật (Technical Constraints)

1. **Ràng buộc công nghệ bắt buộc (Theo mục 1.1 đề bài):**
   - **Frontend:** Next.js 16 (App Router) theo định hướng Mobile-first / PWA.
   - **Backend:** NestJS kiến trúc module/domain hướng đối tượng.
   - **Database:** PostgreSQL 16 (quản lý schema và migrations thông qua Prisma ORM).
   - **Containerization:** Hệ thống đóng gói bằng Docker và khởi chạy được bằng `docker-compose`.
2. **Ràng buộc giao thức & bảo mật:**
   - Giao tiếp Client – Server qua chuẩn RESTful API bằng JSON qua HTTPS.
   - Các API nhạy cảm (tạo đơn, thanh toán, lịch sử, đổi trạng thái) bắt buộc xác thực bằng JWT Bearer Token.
   - Mật khẩu người dùng được băm một chiều an toàn bằng thuật toán `bcrypt` (cost factor ≥ 10).
3. **Ràng buộc toàn vẹn dữ liệu (Data Integrity):**
   - Tạo đơn hàng, lưu chi tiết món, lưu topping và trừ tồn kho phải nằm trong một Database Transaction (Atomic).
   - Áp dụng Khóa lạc quan (*Optimistic Concurrency Control*) bằng trường `version` để chống hiện tượng Race Condition khi trừ kho.

5 Đánh giá rủi ro ban đầu (Initial Risk Assessment)

Bảng 2: Bảng đánh giá rủi ro ban đầu và giải pháp giảm thiểu

Mã	Rủi ro nhận diện	Xác suất	Tác động	Giải pháp giảm thiểu (Mitigation)
R-01	**Bán vượt tồn kho** (*Overselling*)	Trung bình	Rất cao	Áp dụng Optimistic Locking qua cột `version` trong bảng `Product`. Giao dịch xung đột sẽ bị hủy và báo lỗi 400 thân thiện.
R-02	**Thanh toán trùng lặp** (*Double Payment*)	Cao	Nghiêm trọng	Yêu cầu `idempotencyKey` duy nhất trong request thanh toán. Backend kiểm tra bản ghi cũ và trả về kết quả mà không trừ tiền lần 2.
R-03	**Xung đột Schema & DB Migration**	Trung bình	Cao	Sử dụng Prisma Migration có kiểm soát phiên bản bằng Git (`prisma migrate deploy`), tự động chạy an toàn trong pipeline CI/CD trước khi khởi động app.
R-04	**Sự cố triển khai CI/CD Production**	Trung bình	Cao	Chọn nền tảng PaaS hiện đại (Render / Railway / Fly.io) hỗ trợ Docker container, quản lý biến môi trường qua Secret Manager và test sớm từ Sprint 1.